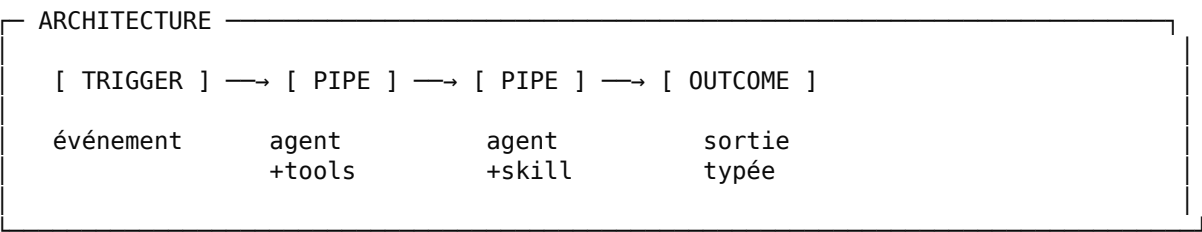


OUVRIER

● WORKERS FOR YOUR APIs.

DOCUMENTATION v0.1

Framework Go pour middlewares agentiques



Pour développeurs Go juniors. Lisez de A à Z, ou piochez par section.
Code minimal, déploiement immédiat, agents prêts pour la production.

SECTION 00 — SOMMAIRE

01	POURQUOI OUVRIER	p. 04
02	INSTALLATION	p. 06
03	VOTRE PREMIER OUVRIER (TUTORIEL)	p. 07
04	LES 4 PRIMITIVES	p. 13
05	TOOLS, SKILLS, MCP	p. 17
06	LA CLI : SCAFFOLD ET MODIFICATION	p. 21
07	CONCURRENCE : PARALLEL, MAP	p. 25
08	DÉPLOIEMENT : SSH ET DOCKER	p. 28
09	MONITORING ET TRACE VIEWER	p. 31
10	RÉFÉRENCE COMPLÈTE DE L'API	p. 33
11	LIMITES ET PIÈGES CONNUS	p. 36
12	GLOSSAIRE	p. 39

— SECTION 01 — POURQUOI OUVRIER

Le problème.

Vous avez un SaaS, un LMS, un CRM. Vous voulez l'augmenter avec un agent IA. Vous avez le choix entre : (a) écrire un plugin natif lourd, (b) bricoler un script Python qui tourne quelque part, (c) utiliser un framework agent généraliste où 80% du code est de la plomberie.

La proposition.

Ouvrier est un framework Go pour scaffolder, builder et déployer des middlewares agentiques en quelques minutes. Vous décrivez un trigger, un goal, des tools. Ouvrier produit un binaire statique prêt à tourner.

MANIFESTE

TROIS PROMESSES

1. CONNECTER un trigger à un outcome
2. PENSER via un agent LLM avec son harnais
3. LIVRER un binaire déployable en une commande

Le mental model.

Pensez Unix. Une commande shell `cat file | grep foo | tee out` compose trois programmes en pipeline. Ouvrier fait pareil avec des agents : `From → Pipe → Pipe → Reply`. Chaque étape consomme la sortie de la précédente. Pas de magie, juste une chaîne de montage.

Quand utiliser Ouvrier.

- ✓ Augmenter un SaaS via webhook ou polling API
- ✓ Automatiser une tâche métier répétitive sur événement
- ✓ Déployer un agent autonome qui tourne en cron
- ✓ Créer un middleware entre un LMS et un LLM
- ✗ Construire une UI conversationnelle (utilisez Vercel AI SDK)
- ✗ Faire du prompt engineering itératif (utilisez un notebook)
- ✗ Orchestrer un workflow business complexe non-IA (Temporal, n8n)

SECTION 02 — INSTALLATION

Prérequis : Go 1.22+ installé. C'est tout.

```
● ● ●
# Installer la CLI Ouvrier
$ go install github.com/yourorg/ouvrier/cmd/ouvrier@latest

# Vérifier
$ ouvrier --version
ouvrier 0.1.0

✓ CLI prête. Tout passe par elle.
```

La CLI ouvrier est la porte d'entrée. Elle scaffold, configure, build, déploie et inspecte. Vous n'avez jamais besoin de toucher au code Go si vous ne le voulez pas.

CLI

COMMANDES PRINCIPALES

ouvrier new	créer un projet interactif
ouvrier add agent	ajouter un agent au pipeline
ouvrier add tool	ajouter un tool Go custom
ouvrier add skill	ajouter une expertise .md
ouvrier dev	lancer en local avec trace viewer
ouvrier build	compiler un binaire statique
ouvrier deploy ssh	déployer en SSH
ouvrier deploy docker	générer image Docker
ouvrier status	santé d'un déploiement
ouvrier trace	inspecter les exécutions

— SECTION 03 — VOTRE PREMIER OUVRIER —

CAS D'USAGE

Vous formez des apprenants sur Moodle. Chaque matin à 6h, un agent doit calculer les cartes de mémorisation dues (algorithme FSRS), générer un message personnalisé, et l'envoyer via la messagerie Moodle.

Étape 1 – Lancer le scaffold



```
$ ouvrier new
```

Une TUI interactive démarre. Style Charm Bracelet : navigation au clavier, couleurs sobres, validation par entrée.

```
TUI · STEP 1/8
┌
│ OUVRIER · NEW PROJECT
│ ┌
│ │ Nom du projet
│ │ > moodle-fsrs-coach_
│ │
│ │ Description courte
│ │ > Recommande des révisions FSRS_
│ │
│ └─┘
│ [↵] valider  [esc] quitter  [tab] suivant
└─┘
```

Étape 2 – Choisir le trigger

Le trigger est l'événement qui déclenche votre pipeline. Quatre choix :

TUI · STEP 2/8

Type de trigger

- | | |
|--|------------------------|
| ☐ HTTP endpoint | POST /chemin/... |
| ☒ Cron (planifié) | expression cron |
| ☐ Webhook signé | avec vérification HMAC |
| ☐ Stream | Kafka, NATS, Redis |

Expression cron

> 0 6 * * * _ tous les jours à 6h

Étape 3 – Définir les agents

Combien d'agents enchaînés ? Pour notre cas : 4. Chacun reçoit la sortie du précédent.

TUI · STEP 3/8 – Agent 1

Agent 1/4

Goal de l'agent

> Récupère la liste des apprenants actifs_

Modèle LLM

- | | |
|------------------------------|--------------------|
| > anthropic/claude-haiku-4-5 | rapide, économique |
| anthropic/claude-sonnet-4-6 | équilibré |
| anthropic/claude-opus-4-7 | raisonnement |
| custom | autre |

Tools (espace pour sélectionner)

- | | |
|--------------------|----------------------|
| [x] tool custom Go | moodle_list_learners |
| [] MCP server | |
| [] bash sandbox | |

Skill (.md) ? > non

La CLI répète pour les agents 2, 3 et 4. Pour l'agent 3 (génération du message personnalisé), vous choisirez Sonnet avec un Skill personalized-message.

Étape 4 – Écrire l'expertise (Skill)

Quand vous ajoutez un Skill, la CLI ouvre un éditeur multi-lignes pour rédiger votre expertise métier en Markdown. C'est ce qui distingue Ouvrier : l'expertise vit dans des fichiers .md, éditables par un pédagogue ou un juriste, pas seulement par un dev.

TUI · STEP 5/8 – Skill personalized-message

Édition du SKILL.md (Ctrl+S pour sauvegarder)

Génération de message de rappel

Tu rédiges un message court (3-5 phrases) que l'apprenant reçoit dans sa messagerie Moodle.

Règles de rédaction

1. Toujours dans la langue de l'apprenant
2. Ton adapté au niveau CEFR (A1-C2)
3. Mentionner le nombre exact de cartes_

Étape 5 – Configurer la sortie

Trois grandes catégories. La CLI choisit Sink par défaut quand le trigger est asynchrone (cron, stream) car personne n'attend de réponse.

TUI · STEP 6/8

Type de sortie

Reply	répondre à l'expéditeur (trigger HTTP)
Push	pousser vers un autre endpoint
(●) Sink	fin sans retour (cron, fire-and-forget)

Format pour Sink

(●) Log uniquement	+ métriques OTel
() File	écrit sur disque

Étape 6 – Renseigner les secrets (.env)

La CLI détecte automatiquement les secrets nécessaires d'après vos choix : clé Anthropic, URL Moodle, token Moodle. Vous les saisissez sécurisé.

```
TUI · STEP 7/8 – Secrets
```

```
Secrets détectés (.env)
```

```
ANTHROPIC_API_KEY    > sk-ant-*****_  
                      saisie masquée
```

```
MOODLE_BASE_URL      > https://lms.acme-formation.fr_  
MOODLE_TOKEN         > *****_  
                      saisie masquée
```

```
✓ .env créé avec permissions 0600  
✓ .env ajouté à .gitignore
```

Étape 7 – Cible de déploiement

```
TUI · STEP 8/8 – Déploiement
```

```
Cible de déploiement principale
```

(●) SSH	binaire + systemd sur VPS
() Docker	image OCI à pousser
() Les deux	génère config pour ssh + docker

```
Host SSH
```

```
> ops@server.acme-formation.fr_
```

```
Chemin sur le serveur
```

```
> /opt/moodle-fsrs-coach_
```


Étape 8 – Génération

```
✓ Projet généré dans ./moodle-fsrs-coach/

./moodle-fsrs-coach/
├── main.go          4 agents générés
├── tools/
│   └── moodle.go    3 tools stub
├── skills/
│   └── personalized-message/
│       └── SKILL.md  votre expertise
├── pip.yaml         config déploiement
├── .env             secrets (0600)
├── .env.example     template
├── .gitignore
├── go.mod
└── README.md        généré pour vous

Prochaines étapes :
cd moodle-fsrs-coach
ouvrier dev          tester localement
ouvrier deploy       mettre en production
```

Étape 9 – Compléter les tools Go

La CLI a généré des stubs pour vos tools custom. Vous devez maintenant les implémenter – c'est la seule étape où vous écrivez du Go. Ouvrez tools/moodle.go :

```
package tools go

import "context"

type Learner struct {
    ID      string `json:"id"`
    Name    string `json:"name"`
    Locale  string `json:"locale"`
}

// ListActiveLearners retourne les apprenants actifs.
// TODO: implémenter l'appel à l'API Moodle.
func ListActiveLearners(ctx context.Context, days int) ([]Learner, error) {
    // ... votre logique ici ...
    return nil, nil
}
```

Étape 10 – Lancer en local

```
● ● ●
$ ouvrier dev

✓ Chargement skills (1)
✓ Chargement tools (3)
✓ Validation pipeline : Cron → 4 Pipes → Log
✓ Cron enregistré, prochain run dans 14h 23m

Listening on http://localhost:8080
GET /admin/health
GET /dev          trace viewer
POST /admin/trigger force-run manuel

✂ Hot-reload actif (main.go, tools/, skills/)
```

Forcer l'exécution sans attendre le cron :

```
● ● ●
$ curl -X POST http://localhost:8080/admin/trigger
{"status": "started", "execution_id": "exec_abc123"}

$ ouvrier trace --last 1
exec_abc123 ✓ 42.3s $0.18
  Pipe 1  Récupère apprenants  ✓ 1.1s  12 learners
  Pipe 2  Calcule cartes FSRS  ✓ 18.4s  87 cards
  Pipe 3  Génère messages      ✓ 19.2s  12 msgs
  Pipe 4  Envoie via Moodle    ✓ 3.6s  12/12 OK
```

Étape 11 – Déployer

```
● ● ●
$ ouvrier deploy ssh

✓ Build pour linux/amd64...
✓ Binaire : 13.2 MB (skills embedded)
✓ Connexion à ops@server.acme-formation.fr...
✓ Upload binaire + .env
✓ Restart moodle-fsrs-coach.service
✓ Health check : 200 OK

Déployé v0.1.0
Prochain cron : demain 06:00 UTC
```

C'est fait. Votre Ouvrier tourne, fait son boulot, vous reviendrez demain.

— SECTION 04 — LES 4 PRIMITIVES

Tout Ouvrier se résume à quatre fonctions. Si vous comprenez ces quatre, vous comprenez 90% du framework.

PRIMITIVE 1/4

From – l'entrée

Le trigger. Quel événement déclenche le pipeline. HTTP, cron, webhook, stream.

```
ovr.From("POST /learners/{id}/due")
ovr.From(ovr.Cron("0 6 * * *"))
ovr.From(ovr.Webhook("stripe"))
ovr.From(ovr.Stream("kafka://events"))
```

[go](#)

PRIMITIVE 2/4

Pipe – l'agent

Un agent défini par un goal en langage naturel. Reçoit un input, raisonne avec son LLM et ses tools, produit un outcome qui alimente le Pipe suivant.

```
ovr.Pipe("Génère un message de rappel personnalisé",
  ovr.Model("anthropic/claude-sonnet-4-6"),
  ovr.Skill("personalized-message"),
  ovr.Tool("send_message", tools.SendMessage),
)
```

[go](#)

PRIMITIVE 3/4

Run – le démarrage

Démarre le serveur HTTP, enregistre les crons, lance les workers de stream. Tous les pipelines sont déclarés dans un seul Run.

```
ovr.Run(":8080",
    ovr.From(...),
    ovr.Pipe(...),
    ovr.Pipe(...),
    ovr.Reply(...), // ou Push(...), ou Sink(...)
)
```

go

PRIMITIVE 4/4

Reply, Push, Sink – la sortie

Trois verbes selon ce que vous voulez faire de l'outcome final. Reply répond à l'expéditeur (trigger HTTP synchrone). Push envoie vers un autre endpoint (webhook, queue). Sink termine sans retour (cron, fire-and-forget).

```
// Trigger HTTP synchrone : on répond à l'appelant
ovr.Reply(ovr.JSON[Result]()) // réponse JSON typée
ovr.Reply(ovr.SSE())          // streaming temps réel
ovr.Reply(ovr.Accepted())      // 202 + traitement asynchrone

// Pousser le résultat ailleurs (après ou en parallèle)
ovr.Push(ovr.Webhook("https://..."))
ovr.Push(ovr.Queue("nats://..."))

// Fin du pipeline, rien à retourner
ovr.Sink(ovr.Log())            // log + métriques
ovr.Sink(ovr.File("./out.json")) // disque
```

go

QUAND

CHOISIR LA SORTIE

Trigger HTTP, l'appelant attend	→ Reply
Trigger async ou résultat à pousser	→ Push
Trigger async, aucun retour attendu	→ Sink

MÉMO

RÈGLE D'OR

Vous n'avez besoin de RIEN d'autre pour 90% des cas.
Les sections suivantes ne sont utiles que pour le reste.

Les deux patterns que vous verrez 90% du temps

Tout pipeline Ouvrier ressemble à l'un de ces deux squelettes. Mémorisez-les, vous saurez écrire 90% des cas réels.

PATTERN 1 – REQUÊTE / RÉPONSE

Une requête HTTP arrive, vos agents font le travail, vous répondez à l'expéditeur. Idéal pour des endpoints d'enrichissement, de classification, de triage.

```
ovr.Run(":8080",  
    ovr.From("POST /tickets/triage"),  
  
    ovr.Pipe("Récupère le contexte du ticket", ...),  
    ovr.Pipe("Classifie et résume", ...),  
  
    // Réponse renvoyée à l'expéditeur HTTP  
    ovr.Reply(ovr.JSON[TriageResult]()),  
)
```

PATTERN 2 – TÂCHE PROGRAMMÉE / FIRE-AND-FORGET

Un cron ou un stream déclenche le pipeline. Personne n'attend de réponse. Le dernier Pipe pousse le résultat vers le service tiers (Moodle, Slack, votre CRM) via un tool. Sink Log marque la fin du pipeline.

```
ovr.Run(":8080",  
    ovr.From(ovr.Cron("0 6 * * *")),  
  
    ovr.Pipe("Récupère les apprenants actifs", ...),  
    ovr.Pipe("Calcule les cartes FSRS dues", ...),  
    ovr.Pipe("Génère un message personnalisé", ...),  
  
    // Side-effect métier : le tool envoie vers Moodle  
    ovr.Pipe("Envoie le message via Moodle",  
        ovr.Tool("moodle_send_message", tools.SendMessage),  
    ),  
  
    // Fin du pipeline – rien à retourner, juste logger  
    ovr.Sink(ovr.Log()),  
)
```

Note importante. Dans le pattern 2, l'envoi à Moodle est un side-effect d'un Pipe (via son tool), pas la primitive de sortie. Sink(Log) dit juste : pipeline terminé, rien à renvoyer côté trigger.

— SECTION 05 – TOOLS, SKILLS, MCP —

Un Pipe sans capacités est limité au texte brut. Pour qu'il agisse, donnez-lui des outils. Ouvrier en propose trois types, complémentaires.

TOOL	SKILL	MCP
Fonction Go	Dossier .md	Serveur externe
Code déterministe Calcul, DB, API HTTP	Expertise métier Instructions LLM	Intégration tierce Standard Anthropic
Coût: 0 Latence: ms	Coût: tokens Latence: 2-10s	Coût: tokens Latence: réseau

Tool – fonction Go

Une fonction Go normale. Ouvrier infère son JSON schema via reflection au démarrage. Utilisez-le pour tout ce qui est calcul, accès DB, ou appel API simple.

```
// Dans tools/moodle.go
func ListActiveLearners(ctx context.Context, days int) ([]Learner, error) {
    // ... appel API Moodle ...
}

// Dans main.go
ovr.Pipe("Récupère les apprenants",
    ovr.Tool("list_learners", tools.ListActiveLearners),
)
```

Skill – expertise en Markdown

Un Skill est un dossier `./skills/nom-skill/` contenant un `SKILL.md` au format Anthropic Agent Skills. C'est là que vit l'expertise métier – éditée par un non-développeur (pédagogue, juriste, support manager).

```
---
name: personalized-message
description: Rédige un message de rappel adapté au niveau et à la langue.
---

# Stratégie de rédaction

Tu rédiges un message court (3-5 phrases) que l'apprenant
reçoit dans sa messagerie Moodle le matin.

## Règles

1. Toujours dans la langue de l'apprenant
2. Ton adapté au niveau CEFR (A1-C2)
3. Mentionner le nombre exact de cartes
4. Pas de markdown, emojis sobres uniquement

## Format de sortie

```json
{ "subject": "...", "body": "..." }
```
```

MCP – intégration tierce

Un MCP server expose un ensemble de tools standardisés via le protocole Model Context Protocol d'Anthropic. Ouvrier s'y connecte comme client. Idéal pour les intégrations SaaS connues (GitHub, Slack, Stripe, etc.).

```
ovr.Pipe("Crée une issue GitHub",
  ovr.MCP("github-mcp"), // URL dans .env : GITHUB_MCP_URL
)
```

CHOISIR

RÈGLE PRATIQUE

Code déterministe (DB, API simple) → Tool
 Procédure / méthodologie rédigée → Skill
 SaaS tiers déjà standardisé → MCP

— SECTION 06 — LA CLI : SCAFFOLD ET MODIFICATION

Vous avez vu `ouvrier new`. La CLI fait beaucoup plus : vous pouvez ajouter, modifier, supprimer des agents, tools, skills sans toucher au code.

Ajouter un agent au pipeline existant

```
● ● ●
$ cd moodle-fsrs-coach
$ ouvrier add agent

Position dans le pipeline
  > après Pipe 2 (Calcule cartes dues)
    après Pipe 3 (Génère messages)
    à la fin

Goal
  > Filtre les apprenants inactifs depuis 30j_
  ...
✓ Agent inséré dans main.go
✓ Pipeline revalidé
```

Ajouter un Skill

```
● ● ●
$ ouvrier add skill

Nom du skill (kebab-case)
  > legal-compliance_

Description courte
  > Vérifie la conformité RGPD du cours soumis_

[ouverture de l'éditeur multi-lignes pour rédiger le SKILL.md]

✓ ./skills/legal-compliance/SKILL.md créé
✓ Disponible via ovr.Skill("legal-compliance")
```


Ajouter un Tool Go

```

$ ouvrier add tool

Nom du tool
> check_copyright_

Description
> Vérifie le copyright d'un texte via l'API Copyleaks_

Signature (entrées)
> text string, lang string_

Signature (sortie)
> CopyrightResult, error_

✓ Stub généré dans tools/copyright.go
  Implémentez la logique. Le schema JSON est auto-généré.

```

Visualiser le pipeline actuel

```

$ ouvrier show

moodle-fsrs-coach · v0.1.0

[ Cron(0 6 * * *) ]
  |
  ▼
[ Pipe 1 ] Récupère apprenants actifs
  └─ Haiku · list_learners (Go tool)
    |
    ▼
[ Pipe 2 ] Calcule cartes FSRS dues
  └─ Haiku · fsrs_compute_due · fetch_cards
    |
    ▼
[ Pipe 3 ] Génère messages personnalisés
  └─ Sonnet · Skill: personalized-message
    |
    ▼
[ Pipe 4 ] Envoie via Moodle
  └─ Haiku · tool: moodle_send_message ← side-effect
    |
    ▼
[ Sink: Log ] ← cron, fire-and-forget

```

Vous pouvez toujours ouvrir `main.go` dans votre éditeur préféré et modifier manuellement. Ouvrier ne vous enferme jamais dans la CLI : le code généré est lisible, idiomatique Go, et n'utilise pas de magie cachée.

— SECTION 07 — CONCURRENCE : PARALLEL, MAP —

Par défaut, les Pipes s'exécutent en séquence : Pipe 2 attend Pipe 1, etc. Pour 20% des cas vous voulez du parallélisme. Trois primitives le permettent.

Parallel – fan-out

Plusieurs Pipes reçoivent le même input et s'exécutent en parallèle. Le Pipe suivant reçoit un tableau des outcomes dans l'ordre déclaré.

```
ovr.Run(":8080",  
    ovr.From("POST /courses/submitted"),  
  
    ovr.Parallel(  
        ovr.Pipe("Évalue la qualité pédagogique", ...),  
        ovr.Pipe("Vérifie la conformité légale", ...),  
        ovr.Pipe("Évalue le niveau CEFR", ...),  
    ),  
  
    ovr.Pipe("Agrège les 3 verdicts en décision finale", ...),  
    ovr.Reply(ovr.JSON[Verdict]()),  
)
```

Map – appliquer sur collection

Le Pipe précédent a produit une liste. Vous voulez traiter chaque élément en parallèle, avec un parallélisme borné.

```
ovr.Pipe("Récupère les apprenants actifs", ...),  
  
ovr.Map(  
    ovr.Concurrency(10), // max 10 en parallèle  
    ovr.Pipe("Calcule cartes dues", ...),  
    ovr.Pipe("Génère message", ...),  
    ovr.Pipe("Envoie", ...),  
)  
  
ovr.Sink(ovr.Log()),
```

WorkerPool – concurrence sur stream

Sur un trigger stream ou webhook à forte volumétrie, borner le nombre de pipelines en vol simultanément.

```
ovr.Run(":8080",  
    ovr.From("POST /webhooks/stripe",  
        ovr.WorkerPool(20), // jusqu'à 20 webhooks simultanés  
    ),  
    ovr.Pipe("Traite la dispute", ...),  
    ovr.Reply(ovr.JSON[Result]()),  
)
```

[go](#)

DÉCIDER

QUAND UTILISER QUOI

| | |
|-------------------|--|
| Pipes séquentiels | A → B → C, dépendances en chaîne |
| Parallel | 3 analyses indépendantes du même input |
| Map | même traitement sur N éléments |
| WorkerPool | borner la concurrence sur stream |

RÈGLE : ne jamais paralléliser ce qui n'a pas besoin de l'être. Le séquentiel est plus simple à débbugger.

Note sur les sub-agents : un Pipe peut invoquer un autre Pipe comme tool via `ovr.SubAgent("name", pipeline)`. Le LLM parent décide quand l'appeler. Cap dur de 5 invocations parallèles par défaut (protège vos rate limits).

— SECTION 08 — DÉPLOIEMENT : SSH ET DOCKER —

Deux cibles supportées en v0.1 : SSH (binaire + systemd sur un VPS) et Docker (image OCI). Choisissez selon votre infrastructure.

SSH – binaire statique

Le binaire Ouvrier est statique et auto-suffisant : pas de runtime, pas de dépendances, pas de fichiers externes. Le SKILL.md et les ressources sont embedded via go:embed.

```
● ● ●
$ ouvrier deploy ssh

✓ Build linux/amd64 statique
✓ Binaire : 13.2 MB
✓ Connexion SSH...
✓ scp binaire + .env (chmod 600)
✓ systemctl restart ouvrier-moodle-fsrs.service
✓ Health check : 200 OK
✓ Tail logs (5s)...
  [Cron] Next run: 2026-05-19 06:00:00 UTC

Déployé en 12.4s
```

Configuration dans pip.yaml (généré par la CLI) :

```
deploy:
  ssh:
    host: ops@server.acme-formation.fr
    path: /opt/moodle-fsrs-coach
    service: moodle-fsrs-coach.service
    healthcheck:
      path: /admin/health
      timeout: 30s
```

yaml

Docker – image OCI

Pour Kubernetes, Fly.io, Cloud Run, ou tout orchestrateur acceptant des images OCI. Ouvrier génère un Dockerfile distroless minimal.

```
● ● ●
$ ouvrier deploy docker

✓ Dockerfile distroless généré
✓ Build image registry.acme.com/moodle-fsrs:0.1.0
✓ Taille image : 14.1 MB
✓ Push vers registry

Étapes suivantes :
  kubectl set image deploy/moodle-fsrs app=registry.acme.com/...
  fly deploy --image registry.acme.com/moodle-fsrs:0.1.0
  docker run -p 8080:8080 --env-file .env registry.acme.com/...
```

SÉCURITÉ

GESTION DES SECRETS

| | |
|----------|---|
| .env | jamais commité, monté à la cible |
| pip.yaml | commité, config de build et deploy |
| binaire | contient le code + skills, AUCUN secret |

En SSH .env est uploadé en chmod 0600

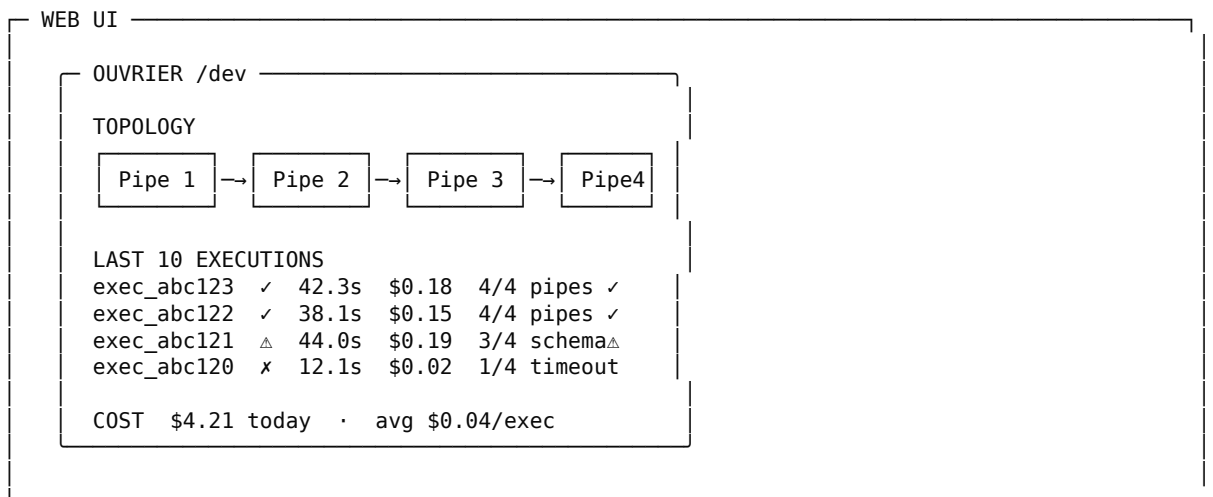
En Docker secrets via --env-file ou orchestrator

SECTION 09 – MONITORING ET TRACE VIEWER

Vous m'avez demandé deux métriques essentielles : santé fonctionnelle (chaque exécution traverse tous les Pipes sans erreur) et santé qualité (les outcomes respectent leur schéma).

Trace viewer en local

ouvrier dev expose `http://localhost:8080/dev`. UI web autonome qui montre la topologie, les exécutions live, les inputs/outputs par Pipe, les coûts LLM.



Inspection à distance

```

$ ouvrier status

moodle-fsrs-coach v0.1.0 · prod.acme.fr:8080
Uptime: 3d 4h

Last 1000 executions
Functional health  994/1000 (99.4%)
Schema conformance 998/1000 (99.8%)
Cost               $4.21 total · $0.0042 avg

Recent failures
12m ago CRM 503 (retried 3x, then failed)
2h ago  schema violation (priority="critical")
```

— SECTION 10 — RÉFÉRENCE COMPLÈTE DE L'API —

Référence exhaustive. Si vous cherchez une fonction, elle est ici.

— TRIGGERS —

From("METHOD /path")

Endpoint HTTP

Cron("0 6 * * *")

Planificateur cron

Webhook("provider")

Webhook signé

Stream("kafka://...")

Stream Kafka/NATS/Redis

— PIPES ET COMPOSITION —

Pipe("goal", opts...)

Agent simple

Parallel(pipes...)

Fan-out parallèle

Map(opts, pipes...)

Application sur collection

SubAgent("name", pipe)

Pipeline comme tool

— CONFIGURATION D'UN PIPE —

Model("provider/model")

Modèle LLM

Tool("name", goFunc)

Tool Go custom

Skill("dir-name")

Dossier .md

MCP("server-name")

MCP server

Bash(Sandbox(path))

Shell sandboxé

— SORTIES — REPLY (HTTP SYNCHRONE) —

Reply(JSON[T]())

Réponse JSON typée à l'appelant

Reply(SSE())

Streaming SSE

Reply(Accepted())

HTTP 202, traitement asynchrone

— SORTIES – PUSH (VERS UN TIERS) —**Push(Webhook(url))**

POST HTTP sortant

Push(Queue(url))

Publish NATS/Kafka/SQS

— SORTIES – SINK (FIN DE PIPELINE) —**Sink(Log())**

Fire-and-forget

Sink(File(path))

Écrit sur disque

— CONCURRENCE ET LIMITES —**Concurrency(N)**

Limite Map

WorkerPool(N)

Limite stream

Timeout(d)

Borne temporelle

Retry(n, backoff)

Politique de retry

— SYSTÈME —**Run(":port", ...)**

Démarre le serveur

RequireEnv(vars...)

Validation env au boot

IdempotencyKey(header)

Clé d'idempotence

— SECTION 11 — LIMITES ET PIÈGES CONNUS

Honnêteté avant tout. Ouvrier est un outil, pas une baguette magique. Voici ce qu'il ne fait pas bien.

x LLM non-déterministes

Ouvrier valide les outcomes (schéma JSON conforme), pas le raisonnement. Un Pipe peut produire deux résultats différents sur deux appels. Préparez vos tests en conséquence.

x Pas pour l'expérimentation

Ouvrier est un framework de production. Si vous itérez sur des prompts, utilisez un notebook Python à côté, validez, puis portez dans Ouvrier.

x Écosystème Go vs Python/TS

Moins d'exemples StackOverflow, moins de MCP servers communautaires. Si vous bloquez, vous débinez plus seul que dans l'écosystème Python.

x Secrets en SSH simple

Le .env est uploadé en clair sur le serveur (chmod 0600). Pour la prod sérieuse multi-clients, intégrez Vault, SOPS ou Doppler – pas fourni en v0.

x Courbe d'apprentissage cumulée

Chaque concept est simple : Go, LLM tokens, JSON schemas, MCP, .md, sandbox, deploy. La somme demande un dev backend intermédiaire, pas un débutant absolu.

x Pas de visual builder

La CLI est interactive (TUI), mais ce n'est pas du drag-drop. Si votre cible inclut des non-codeurs, prévoyez une UI web par-dessus qui génère du Go.

x Optimisations conjoncturelles

Prompt caching et parallel tool calling, intégrés par Ouvrier, finiront commodifiés par les SDKs officiels. Votre avantage économique est temporaire.

— SECTION 12 — GLOSSAIRE —

Définitions de tous les termes utilisés dans cette documentation. Classés par catégorie. Si un mot vous est inconnu, il est ici.

— LES 4 PRIMITIVES OUVRIER —

From

Le trigger d'entrée d'un pipeline Ouvrier. Définit ce qui déclenche l'exécution : une requête HTTP, un cron, un webhook signé, ou un stream. C'est toujours le premier argument de Run.

Pipe

Une étape du pipeline. Un agent autonome défini par un goal en langage naturel, un modèle LLM, et un ensemble de tools / skills / MCP. Reçoit l'outcome du Pipe précédent, raisonne, produit le sien.

Run

La fonction qui démarre le serveur HTTP, enregistre les crons, lance les workers de stream. Tous les pipelines sont déclarés à l'intérieur d'un seul Run dans main.go.

Reply

Sortie qui répond à l'expéditeur d'une requête HTTP. À utiliser quand le trigger est HTTP synchrone et qu'on veut renvoyer une réponse (JSON, SSE, 202 Accepted).

Push

Sortie qui pousse l'outcome vers un autre endpoint (webhook sortant, queue NATS/Kafka). Asynchrone. À utiliser quand le résultat doit atterrir ailleurs que chez l'appelant.

Sink

Sortie qui termine le pipeline sans rien renvoyer. À utiliser quand le trigger est asynchrone (cron, stream) et que personne n'attend une réponse. Variantes : Log, File.

— COMPOSANTS D'UN PIPE —

Goal

L'objectif d'un Pipe, rédigé en langage naturel. C'est le contrat de l'agent : ce qu'il doit accomplir. Exemple : « Récupère l'état FSRS de l'apprenant et identifie

les cartes dues ». Le LLM raisonne pour atteindre ce goal.

Model

Le modèle LLM utilisé par un Pipe. Format provider/model. Exemple : anthropic/claude-sonnet-4-6. Toujours explicite, jamais d'alias magique.

Tool

Une fonction Go enregistrée comme capacité d'un agent. Déterministe, rapide (microsecondes à millisecondes), coût négligeable. Utilisé pour tout ce qui est calcul, accès base de données, ou appel API simple.

Skill

Un dossier contenant un fichier SKILL.md au format Anthropic Agent Skills. Frontmatter (name, description) + corps Markdown qui contient les instructions métier. Édité par l'expert du domaine, pas le dev.

MCP

Model Context Protocol. Standard ouvert d'Anthropic pour exposer des tools via un serveur. Un MCP server peut être consommé par n'importe quel agent. Ouvrier s'y connecte comme client.

Harnais

L'infrastructure invisible qui entoure un Pipe : tool-use loop, retry, prompt caching, parallel tool calling, observabilité, gestion des erreurs. Ouvrier le fournit, vous n'écrivez rien.

Tool-use loop

Le cycle interne d'un agent : il reçoit un prompt, peut décider d'appeler des tools, reçoit les résultats, raisonne à nouveau, peut appeler d'autres tools, jusqu'à produire son outcome final.

— TRIGGERS — TYPES D'ENTRÉE —

Trigger

L'événement qui déclenche l'exécution d'un pipeline. Concrètement : une requête HTTP entrante, l'heure d'un cron, un message dans une queue, un webhook signé.

HTTP synchrone

Une requête HTTP qui attend une réponse. Le client envoie POST, attend, reçoit le JSON. Couple Reply à un From HTTP.

Cron

Un planificateur de tâches qui exécute le pipeline à intervalles réguliers définis par une expression cron. Exemple : 0 6 * * * = tous les jours à 6h.

Webhook

Une URL HTTP exposée par votre serveur pour recevoir des événements envoyés par un service tiers (Stripe, GitHub, Slack). Souvent signé cryptographiquement pour prouver l'authenticité.

Stream

Un flux continu d'événements depuis Kafka, NATS, Redis Streams, etc. Le pipeline consomme les événements au fur et à mesure.

— CONCURRENCE —

Séquentiel

Mode d'exécution par défaut des Pipes. Pipe 2 attend que Pipe 1 ait terminé avant de démarrer. Simple à comprendre et à déboguer.

Concurrent / parallèle

Mode où plusieurs Pipes s'exécutent en même temps. Plus rapide mais plus complexe : ordering, fail-fast, coordination.

Fan-out

Pattern où un input alimente plusieurs Pipes en parallèle. Métaphore : un éventail (« fan ») qui s'ouvre vers plusieurs branches. Implémenté par `ovr.Parallel`.

Fan-in

Inverse du fan-out : plusieurs outcomes parallèles convergent vers un seul Pipe d'agrégation. Souvent utilisé immédiatement après un fan-out pour synthétiser les résultats.

Map

Appliquer le même sous-pipeline à chaque élément d'une collection, en parallèle, avec un parallélisme borné. Inspiré du map fonctionnel.

WorkerPool

Un nombre fixe de workers qui consomment une file d'événements. Borne la concurrence pour ne pas saturer les rate limits du LLM ou de l'API cible.

Concurrency

Le nombre maximum d'opérations en parallèle. Souvent borné pour respecter les limites du LLM (rate limit Anthropic, OpenAI, etc.).

Backpressure

Mécanisme qui ralentit la production d'événements quand le pipeline ne suit plus le rythme. Évite l'explosion mémoire et la perte de messages.

— DÉPLOIEMENT ET FIABILITÉ —

Binaire statique

Un exécutable Go qui contient tout : code, dépendances, ressources embarquées. Pas besoin d'un runtime installé. Copiable d'une machine à l'autre.

go:embed

Directive Go qui embarque des fichiers dans le binaire à la compilation. Ouvrier l'utilise pour les SKILL.md : votre binaire contient son expertise.

Distroless

Image Docker minimaliste qui ne contient que les fichiers strictement nécessaires (pas de shell, pas d'apt, pas de gestionnaire de paquets). Surface d'attaque réduite, taille minimale.

Systemd

Le gestionnaire de services standard de Linux moderne. Ouvrier génère une unité systemd quand vous déployez en SSH.

Healthcheck

Endpoint HTTP (par défaut /admin/health) qui retourne 200 OK quand le service est opérationnel. Utilisé par les load balancers et les orchestrateurs pour décider si le service est vivant.

Retry

Réessayer une opération qui a échoué. Ouvrier retry automatiquement sur erreurs transitoires (HTTP 5xx, timeout réseau, rate limit), avec backoff exponentiel.

Backoff exponentiel

Stratégie de retry où le délai entre tentatives double à chaque fois : 1s, 2s, 4s, 8s... Évite de marteler un service en panne.

Idempotency key

Identifiant unique d'une requête qui permet de détecter les doublons. Si Stripe livre deux fois le même webhook, votre pipeline ne s'exécute qu'une fois.

Fire-and-forget

Mode où on déclenche une action sans attendre ni vérifier le résultat. Utile pour des notifications, du logging asynchrone, des tâches background.

— LLM ET COÛTS —

LLM

Large Language Model. Modèle d'IA qui prédit la suite d'un texte. Exemples : Claude (Anthropic), GPT (OpenAI), Gemini (Google).

Token

Unité de découpage d'un texte par un LLM. Approximativement 1 token $\approx$ 4 caractères en anglais, un peu plus en français. Les LLM facturent au token (input + output).

Prompt

Le texte envoyé au LLM. Comprend le system prompt (instructions permanentes) et le user prompt (la question ou la tâche).

System prompt

Instructions permanentes données au LLM en début de conversation. Définit son rôle, son ton, ses contraintes. Dans Ouvrier, c'est généré à partir du goal du Pipe et du SKILL.md.

Prompt caching

Optimisation qui mémorise un préfixe de prompt côté provider LLM. Les appels suivants payent ~10% du prix du préfixe au lieu de 100%. Ouvrier active ça automatiquement.

Parallel tool calling

Capacité d'un LLM moderne à décider d'appeler plusieurs tools en parallèle dans une seule réponse. Divise la latence par N pour les tâches multi-tools indépendantes.

JSON Schema

Format standard pour décrire la structure d'un objet JSON (types, champs obligatoires, valeurs autorisées). Utilisé pour valider les outcomes typés et pour exposer les tools au LLM.

Schema conformance

Le fait qu'un outcome respecte le JSON Schema déclaré. Ouvrier valide chaque sortie. Métrique exposée dans ouvrier status.

Tokens output / input

Tokens en entrée = ce que vous envoyez au LLM. Tokens en sortie = ce qu'il génère. Les deux sont facturés, l'output coûte généralement 3-5x l'input.

— OBSERVABILITÉ —

OTel – OpenTelemetry

Standard d'observabilité (traces, métriques, logs). Ouvrier instrumente automatiquement chaque Pipe. Exportable vers Datadog, Grafana, Honeycomb, etc.

Span

Une unité de travail dans une trace OTel : un appel HTTP, une exécution de Pipe, un tool call. Les spans s'imbriquent pour former une trace complète.

Trace

L'arbre complet des spans pour une exécution. Permet de voir où le temps a été passé, où une erreur a eu lieu.

Trace viewer

L'UI web exposée par ouvrier dev sur /dev. Visualise les exécutions, les coûts, les inputs / outputs de chaque Pipe.

Métriques

Mesures numériques agrégées (latence p50/p95/p99, throughput, taux d'erreur, coût LLM cumulé). Exposées en format Prometheus.

— AGENTS ET PATTERNS AVANCÉS —

Agent

Programme autonome qui reçoit un objectif, raisonne, utilise des outils et produit un résultat. Dans Ouvrier, chaque Pipe est un agent.

Middleware agentique

Petit service autonome (un agent ou un pipeline d'agents) qui s'insère entre un système existant et un LLM pour ajouter de l'intelligence sans modifier le système source.

Sub-agent

Un pipeline Ouvrier exposé comme tool à un autre Pipe. Le LLM parent peut décider de l'invoquer. Permet la composition récursive d'agents.

Side-effect

Effet d'une fonction qui ne se reflète pas dans sa valeur de retour : écriture en base, envoi d'email, modification d'un fichier. Dans le pattern fire-and-forget, le dernier Pipe a souvent un side-effect (envoi à Moodle, push Slack).

Pipeline

Une chaîne de Pipes connectés. Ouvrier supporte les pipelines linéaires (A→B→C) et avec topologies (Parallel, Map).

DAG

Directed Acyclic Graph. Graphe orienté sans cycle. Modèle d'un pipeline complexe avec fan-out / fan-in. Non implémenté en v0.1 d'Ouvrier (uniquement linéaire + Parallel +

Map).

— AUTRES —

.env

Fichier qui contient les secrets et la configuration sensible (clés API, tokens, URLs internes). Jamais commité dans Git. Permissions 0600.

Scaffold

Génération automatique de la structure de fichiers d'un projet (dossiers, fichiers de base, configuration). `ouvrier new` fait du scaffolding interactif.

TUI

Terminal User Interface. Interface utilisateur dans le terminal avec navigation au clavier, couleurs, menus. La CLI Ouvrier est une TUI construite avec Charm Bracelet.

Charm Bracelet

Bibliothèque Go (`charmbracelet/bubbletea`) pour construire des TUIs élégantes. Utilisée par Ouvrier pour l'interface CLI interactive.

Hot-reload

Recompilation et redémarrage automatique du serveur quand vous modifiez un fichier source. Activé par `ouvrier dev`.

Reflection

Capacité de Go à inspecter ses types à l'exécution. Ouvrier l'utilise pour inférer le JSON Schema d'un Tool depuis sa signature Go (paramètres et type de retour).

SSE

Server-Sent Events. Protocole HTTP qui permet à un serveur d'envoyer un flux continu de messages à un client (unidirectionnel). Utilisé pour le streaming d'outcomes.

Boot / Démarrage

Phase où Ouvrier charge les skills, valide les tools, vérifie les variables d'environnement, valide le pipeline. Si quelque chose manque, fail-fast avant d'accepter du trafic.

Fail-fast

Principe de design : signaler une erreur le plus tôt possible plutôt que de laisser le système tourner en état dégradé. Ouvrier applique ça au boot.

Stub

Une implémentation minimale d'une fonction, générée par un outil, qu'il vous reste à compléter. `ouvrier add tool` génère des stubs Go.

FIN DE LA DOCUMENTATION v0.1

Ouvrier est conçu par Arnaud Guiovanna · AI Product Builder · Normandie.
Construit pour livrer vite des middlewares agentiques sur des SaaS existants.
Inspiré par Unix, Anthropic Skills, MCP, et la pratique du tooling interne.

— EOF —